

VD notation

Can Valid

December 2025

1 Introduction

SDVD interface (ADT). State: a named authority $(\text{Name}(D), D)$ with an entry-log $(E_n)_{n \geq 0}$. Operations:

$$\begin{aligned}\text{lookup}(w) &:= D(w), \\ \text{define}(w, \delta)E_n : w &:= \delta \text{ (append)}, \\ D^{(n)} &:= \text{stage}(n), \\ {}_n(w) &:= \exists k < n \text{ s.t. } \text{head}(E_k) = w.\end{aligned}$$

Scope of this paper. We treat the SDVD as an append-only construction: entries are added in a fixed order and no in-place edits are permitted. We therefore do not propose a governance theory of error-correction (renaming, deprecation, merge/split policies, etc.) in this work. If an author wishes to revise an entry, the intended procedure here is to restart the build from a clean log. A separate follow-up will study admissible update policies and their effect on recognisable law-formation.

Important clarifier We need to make sure readers understand that entries to the VD can be made even though the entries are not physical laws. An example might be "mass1 = the mass associated with particle 1." or "spring extension = the amount a spring is displaced by from its equilibrium position. "

SDVD (abstract interface). An SDVD instance consists of a name $\text{Name} \in \text{Strings}$, a map $D : \text{Strings} \rightarrow \text{Strings}$, and an append-only log $(E_n)_{n \geq 0}$.

Operations:

$$\begin{aligned}\text{lookup}(w) &:= D(w), \\ \text{define}(w, \delta)E_n : w &:= \delta, \\ D^{(n)} &:= \text{stage}(n) \quad (\text{replay } E_0, \dots, E_n), \\ {}_n(w) &:= \exists k < n \text{ with } \text{head}(E_k) = w, \\ \text{Triplet}(i, j, k) &:= \text{mutual token-mentions among } (E_i, E_j, E_k).\end{aligned}$$

Concept	Paper notation	Pseudocode	Python
Lookup	$D(w)$	<code>SDVD.D(w)</code>	<code>D(w)</code>
Commit entry	$E_n : w := \delta$	<code>SDVD.define(w,)</code>	<code>define(w,d)</code>
Entry-log view	$(E_n)_{n \geq 0}$	<code>SDVD.print_log()</code>	<code>print_log()</code>
Stage (snapshot)	$D^{(n)}$	<code>SDVD.stage(n)</code>	<code>stage(n)</code>
Previously defined	$_n(w)$	<code>SDVD.previously_defined(w,before=n)</code>	<code>previously_defined(w,before)</code>
Triplet recogniser	(i, j, k)	<code>SDVD.is_triplet(i,j,k)</code>	<code>is_triplet(i,j,k)</code>

Table 1: Core SDVD operations. Pseudocode is shorthand for the formal interface.

We treat all recognition predicates (e.g. *Triplet*) as purely syntactic: they operate on strings.

Multi-valued lookup (polysemy) and why $D : \text{Strings} \rightarrow \text{Strings}$ is too strong. In the early presentation it is tempting to model a dictionary as a function $D : \text{Strings} \rightarrow \text{Strings}$, sending each word w to a unique definition-string $D(w)$. However, this single-valued model is incompatible with the intended SDVD workflow: (i) the construction is *append-only* (entries are added in a linear order), and (ii) a word may legitimately acquire multiple simultaneously active definition-strings (e.g. distinct senses, regimes, approximations, or competing proposals). If we insist on D being a function, then whenever a word is defined twice we must quietly introduce an additional policy (“latest definition wins”, “first definition wins”, “deprecated definitions are inactive”, etc.). Such a policy is precisely the kind of dictionary-modification rule we aim to *avoid* in this paper.

Entry-log as primitive object. We therefore take the primitive object to be the entry-log rather than a single-valued map. An entry is a pair

$$E_n := (w_n, \delta_n) \in \text{Strings} \times \text{Strings},$$

and a stage- n dictionary state is the finite set (or ordered list)

$$\mathcal{D}^{(n)} := \{(w_k, \delta_k) : 0 \leq k \leq n\}.$$

Lookup is induced from the log as a *multi-map* (equivalently, a relation)

$$\mathcal{D}^{(n)}[w] := \{\delta \in \text{Strings} : (w, \delta) \in \mathcal{D}^{(n)}\} \subseteq \text{Strings},$$

or, when order matters, as the list of δ_k with headword $w_k \equiv w$ in increasing log index k .

Why this change is necessary (and not an extra rule). Allowing $\mathcal{D}^{(n)}[w]$ to contain multiple definition-strings does not add a new governance rule; it removes an implicit one. The append-only log already determines *what commitments have been made* (the multiset of entries), and multi-valued lookup

simply reports those commitments without choosing between them. Any subsequent practice of selecting, ranking, or reconciling definitions—for example, a choice function $S^{(n)}(w) \in \mathcal{D}^{(n)}[w]$ or a consistency filter—is a separate layer of methodology, which we postpone.

Consequences for “previously defined” and law-recognition. With the log as primitive, “previously defined” is a statement about headwords in the log:

$$_n(w) : \Longleftrightarrow \exists k < n \text{ such that } w_k \equiv w.$$

Law-recognition likewise remains entry-indexed: a triplet is a property of three entries (E_i, E_j, E_k) , not of a globally unique value $D(w)$. This keeps the formal system faithful to the guiding principle that SDVD commitments are syntactic (strings in an ordered log), while deferring semantic questions of consistency or preferred usage to later work.

Unique-November constraint (one November word per law). In addition to the entry-log and triplet recognition rules, we impose a simple uniqueness constraint on *law anchors*. Each recognised physical law has a designated *November* headword (the previously-logged term that is locked to two newly introduced terms). We require that no headword may serve as the November word for more than one law.

Formally, let $\mathcal{L}$ denote the set of recognised law-events (triplets), and let $\text{Nov} : \mathcal{L} \rightarrow \text{Strings}$ return the November headword of a law-event. The *unique-November* constraint is

$$\forall w \in \text{Strings}, \quad |\{ L \in \mathcal{L} : \text{Nov}(L) \equiv w \}| \leq 1,$$

i.e. Nov is injective. This constraint does *not* prohibit polysemy: a headword may still admit multiple simultaneously active definition-strings in the entry-log. It only prohibits multiple distinct *law-introductions* from being anchored to the same November headword.

Hypergraph interpretation. Represent each law-event $L \in \mathcal{L}$ as a hyper-edge incident to the participating headwords, and mark one incident vertex as the November vertex. The unique-November constraint is equivalent to stating that the November-marked incidence relation forms a matching: each word-vertex has November-degree at most one. This endows the law-subgraph with a clean provenance structure (a unique “anchor” per law) while leaving the full entry-log free to contain additional, non-law definitions of the same headwords.

We will most likely also need to write procedures to police this new rule when in code.

2 Entrenched VD Insights from the Newtonian Case Study

The following are VD-specific *practice rules* and *structural insights* distilled from our attempt to encode Newtonian mechanics. These are not claims about what is true in nature; they are commitments about how we represent and audit a theory in the VD.

1. Separate bookkeeping from attribution.

- *Bookkeeping*: definitions such as `interacting_forces.set` and `net_force` specify how contributions are aggregated once they are included.
- *Attribution*: deciding which contributions are included is a meaning-level recognition step unless a specific mechanism is explicitly declared.
- *Rule*: never allow an aggregate term (e.g. `net_force`) to silently imply a causal source. Causality attribution must be a separate layer.

2. Do not infer interaction from kinematics alone.

- Trajectories that look like collisions do not uniquely determine whether particles interacted; particles can approach and later separate without interaction (e.g. passing through in a toy model).
- Conservation of total momentum for a subsystem does not imply internal interaction; it only implies vanishing net external impulse on that subsystem.
- *Rule*: treat “is interacting” as a modeling decision unless the interaction structure is explicitly represented.

3. Localize irreducible complexity into recognition.

- The hardest part of mechanics is not the algebra but the recognition step: system boundary choice, contact detection, regime selection, and deciding what is “in” the interaction set.
- *Rule*: the only allowed vagueness is in recognition predicates; once recognition inputs are declared, all downstream steps must be mechanically executable and auditable.

4. Use explicit intermediate bridge words to avoid smuggling intuition.

- Newton I becomes structurally clearer when one introduces an explicit bridge term such as `uniform_motion_on_interval(I)` or `uniform_motion_at_time(t0)` rather than relying on tacit acceleration intuition.
- *Rule*: when a law depends on an intuition that readers typically import silently, introduce and lock an explicit bridge word.

5. **Arguments must appear in the headword for function-like terms.**
 - *Rule:* if a term behaves like a function or predicate of parameters (time, particle labels, intervals), include those parameters in the headword (schema style), e.g. `uniform_motion_at_time(t0)` not `uniform_motion_at_time`.
 - Motivation: prevents hidden/free variables and makes the dictionary log auditable.
6. **Triplets are for commitments; raw-meaning entries are for execution semantics.**
 - Triplets lock a stable mutual relationship between terms (e.g. `raw_class`, `class_specific_features`, `instance_of`).
 - Raw-meaning entries specify human-executable procedures (e.g. book-keeping steps, recognition inputs, update rules) without attempting to define interpretability primitives.
 - *Rule:* use triplets when you want immovability; use raw-meaning when you want operational instructions.
7. **Track “degree of law” as a structural dependency metric.**
 - *Definition (informal):* a law is higher-degree if its November term was previously introduced as a winter term of an earlier law; degree measures layering.
 - *Rule:* record degree as part of the audit trail to make compositional structure visible.
8. **Maintain strict control of November words (bridge discipline).**
 - *Rule:* a November word is a deliberate anchor; do not promote a term to November unless you intend to anchor a new commitment.
 - Keep a ledger of November usage to prevent accidental reuse and semantic drift.
9. **Interpretability is assumed; do not over-define.**
 - We do not define basic interpretive operations (e.g. arithmetic, summation, human recognition) inside the VD; we treat them as meaning-level competence, as with $F = ma$ where multiplication is not defined.
 - *Rule:* avoid adding dictionary entries whose only purpose is to justify interpretability; prefer explicit bridge terms only when they remove ambiguity about dependencies.
10. **Distinguish existence from activation (presence vs activity).**

- A modeled entity can exist even when its instantaneous contribution is zero (e.g. a spring exists even if not stretched).
- *Rule:* represent “object exists” separately from “object is active / contributes now,” so that conceptual presence is not conflated with force magnitude.

11. **Avoid inferring causal source from net quantities.**

- `net_force` is an aggregate; equal-and-opposite `net_forces` do not identify whether the source was mutual interaction, mediation by a third object, or boundary exchange.
- *Rule:* causal/source claims require explicit contributory structure (e.g. force-terms or later acting-object instances), not merely net equalities.

12. **System boundaries must be explicit when action–reaction is discussed.**

- Momentum conservation and Newton III style reasoning depend on whether reaction partners are included in the system.
- *Rule:* whenever a reaction term is not represented within the modeled system, the omission must be stated as an explicit boundary/externalization choice.